TrazaCafé Huila: arquitectura orientada a eventos para la trazabilidad del café frente al Reglamento Europeo sobre Deforestación

Proyecto final — Patrones modernos: diseño e implementación de arquitecturas de vanguardia

Victor Andres Rojas Meneses
Especialización en Arquitectura de Software, Corporación Universitaria del Huila (CORHUILA)

Diseño y Patrones Arquitectónicos

Docente: Jesús Ariel González Bonilla

Neiva, 1 de noviembre de 2026

Introducción y justificación

Contexto y problema

Desde el 30 de diciembre de 2026, los operadores medianos y grandes que comercialicen café en la Unión Europea deberán demostrar que el grano no proviene de tierras deforestadas después del 31 de diciembre de 2020 (Reglamento [UE] 2023/1115, 2023). La fecha de aplicación se aplazó dos veces. La última modificación, el Reglamento (UE) 2025/2650, fijó ese plazo y dejó hasta el 30 de junio de 2027 a la mayoría de micro y pequeñas empresas (Reglamento [UE] 2025/2650, 2025). Para el Huila, uno de los principales departamentos cafeteros del país, el asunto no es abstracto: una parte importante del café que se exporta sale de fincas pequeñas, se recibe en puntos de acopio y se mezcla en lotes antes de llegar al exportador.

De ahí partimos. En un acopio rural la conectividad es intermitente, las entregas se anotan en papel o en hojas de cálculo y, cuando el exportador pregunta de qué fincas viene un lote, la respuesta se reconstruye a mano. Esa reconstrucción es lenta y difícil de sostener ante un auditor. Hay además un componente que no es técnico: la información de las fincas pertenece a los caficultores, de modo que compartirla con el exportador exige su autorización, en línea con el régimen colombiano de protección de datos personales (Congreso de la República de Colombia, 2012).

Justificación arquitectónica

Escogimos este problema porque cada uno de sus rasgos se traduce en un atributo de calidad concreto. La conectividad intermitente obliga a que un reenvío no duplique kilos. La auditoría exige que ningún evento se pierda. La verificación geoespacial depende de un proveedor externo, lento y cambiante. Y la regulación sigue en revisión, así que sus reglas deben poder ajustarse sin reescribir el sistema. Con

esos drivers sobre la mesa, la pregunta del curso deja de ser qué patrón moderno usar y pasa a ser qué patrón resuelve qué problema. Ese fue el criterio durante todo el proyecto: un patrón entra al diseño solo si responde a un driver que se pueda medir (Bass et al., 2021; Richards y Ford, 2020).

Objetivo y alcance

El objetivo fue diseñar e implementar un producto mínimo viable (MVP) que registre entregas de café de forma idempotente, conforme lotes de exportación con balance de masa, verifique cada finca contra áreas con pérdida de bosque y entregue la genealogía de cada lote. Esa genealogía es el insumo de la declaración de diligencia debida; la declaración como tal la presenta el operador en la Unión Europea y queda por fuera del alcance. El proyecto se desarrolló de forma individual en un plazo cercano a 24 horas, corre localmente con Docker Compose y usa solo datos sintéticos, por lo que sus resultados no tienen valor probatorio.

El documento sigue el orden en que tomamos las decisiones: primero el diseño arquitectónico (drivers, vistas, decisiones y tecnologías), luego la implementación del MVP, después su evaluación frente a dos alternativas y, por último, las conclusiones y los aprendizajes.

Diseño arquitectónico

Drivers arquitectónicos

El MVP tiene cuatro requisitos funcionales: sincronizar entregas registradas sin conexión (RF1), conformar un lote con balance de masa (RF2), verificar el lote de forma asíncrona contra pérdidas de bosque (RF3) y consultar su genealogía (RF4). Las restricciones pesaron tanto como los requisitos: un solo desarrollador, un plazo cercano

a 24 horas, despliegue local, .NET 10, PostgreSQL y RabbitMQ, y ningún acceso real al sistema de información de la Unión Europea ni a un proveedor geoespacial. Los atributos de calidad los escribimos como escenarios de estímulo, respuesta y medida, siguiendo a Bass et al. (2021), para poder comprobarlos después con datos (Tabla 1).

Tabla 1

Escenarios de atributos de calidad del sistema

Atributo	Estímulo	Respuesta	Medida
QA1 · Integridad	El acopio reenvía entregas ya sincronizadas, idénticas o con contenido alterado.	Reenvío idéntico: 200 sin efectos. Mismo identificador con contenido distinto: 409 y no se guarda nada.	0 kilos duplicados en el 100 % de los reenvíos.
QA2 · Disponibilidad y aislamiento de fallas	Cumplimiento está caído cuando el analista conforma un lote.	La API responde 202; el evento espera y se procesa cuando Cumplimiento se recupera.	0 eventos perdidos; estado final en ≤ 10 s tras la recuperación.
QA3 · Auditabilidad	Un auditor pide la genealogía de un lote.	Lote, fincas, entregas y resultado de verificación, con marcas de tiempo.	100 % de los kilos trazables; respuesta < 500 ms con 200 entregas.
QA4 · Modificabilidad	Se cambia el proveedor de verificación geoespacial.	Se agrega un adaptador que implementa el puerto existente.	0 cambios en los proyectos Domain y Application.

Nota. Elaboración propia con el formato de escenarios de Bass et al. (2021).

Del atributo de calidad al patrón

La cadena de decisión fue esta. QA1 se resolvió con idempotencia: cada entrega lleva un identificador generado en el cliente y un hash SHA-256 de su contenido canónico. QA2 pedía que el analista no dependiera de la disponibilidad del verificador, así que la verificación se volvió asíncrona y orientada a eventos. Para que la asincronía no abriera la puerta a eventos perdidos usamos el patrón Transactional Outbox, que

guarda el evento en la misma transacción que el cambio de negocio, y un inbox en el consumidor para que los mensajes repetidos no repitan efectos (Richardson, 2018). QA4 llevó a la arquitectura hexagonal: el dominio y los casos de uso no conocen EF Core, RabbitMQ ni NetTopologySuite, y cada tecnología entra por un adaptador (Cockburn, 2005). La consulta de genealogía (QA3) tiene una forma muy distinta a la de las escrituras, por lo que separamos comandos y consultas en un CQRS ligero: las escrituras pasan por el agregado y las lecturas van directo a una vista SQL (Fowler, 2011).

También descartamos patrones de forma explícita. No usamos Event Sourcing, porque reconstruir agregados a partir de eventos añade una complejidad que ningún driver pedía y el diario de auditoría ya lo aporta el outbox. Tampoco partimos de Serverless: en un despliegue local no había picos de carga que lo justificaran. Lo retomamos en la evaluación como alternativa.

Contextos acotados y modelo de dominio

Siguiendo a Evans (2003) y Vernon (2013), dividimos el sistema en dos contextos acotados. Acopio y Lotes es el núcleo y maneja fincas, entregas y lotes. Cumplimiento es un contexto de soporte: recibe los lotes, consulta las pérdidas de bosque y devuelve un resultado por finca. Los dos se comunican solo mediante eventos versionados (FincaRegistrada, LoteConformado y LoteVerificado) y cada uno tiene su propia base de datos lógica. La separación tiene una razón concreta: Cumplimiento depende de un servicio externo lento y su carga crece con el número de fincas, no con el de entregas.

El agregado Lote concentra las reglas que un auditor va a revisar. Tiene cinco invariantes: al menos una entrega (I1); una entrega pertenece como máximo a un lote

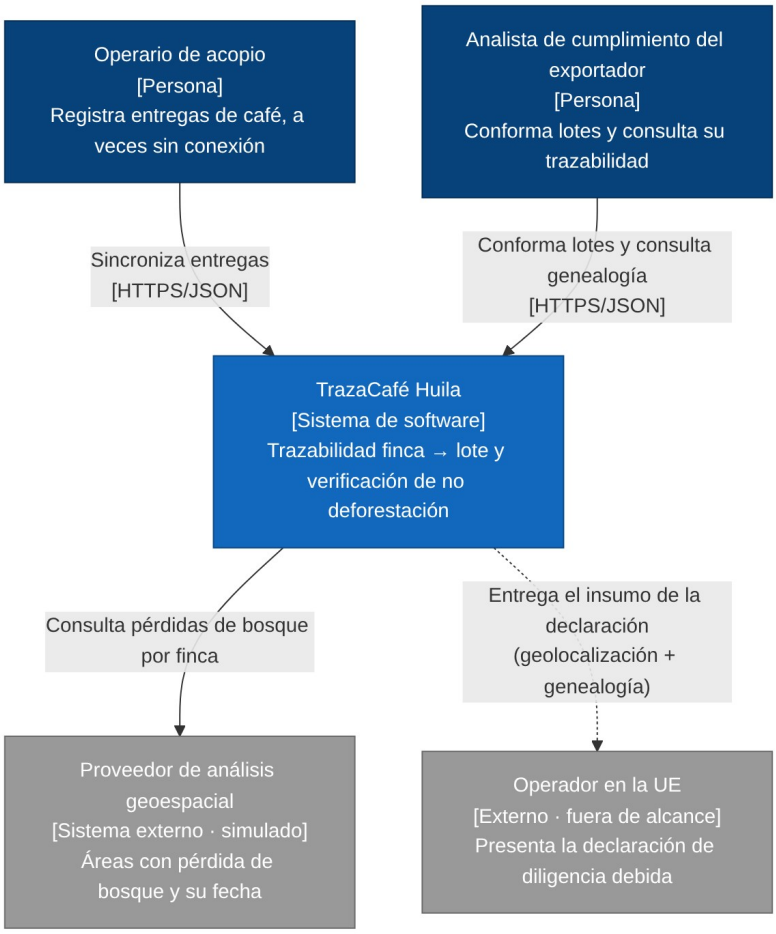
(I2); los kilos del lote son la suma de sus entregas, es decir, el balance de masa (I3); solo se transita desde PendienteVerificacion y los estados finales no cambian (I4); y basta una finca no conforme para que el lote quede Bloqueado (I5). A ellas sumamos una regla que salió del contexto colombiano: no entra al lote ninguna finca cuyo caficultor no haya autorizado el uso de sus datos. En Cumplimiento, la regla de la fecha de corte vive en el dominio y no en el adaptador geoespacial; así, cambiar de proveedor no puede alterarla por accidente.

Vistas de la arquitectura

Documentamos la arquitectura con el modelo C4 (Brown, s.f.) y con diagramas UML de despliegue y de secuencia, escritos en Mermaid y versionados junto al código. La Figura 1 muestra el contexto: el operario de acopio y el analista del exportador usan TrazaCafé Huila, que consulta a un proveedor geoespacial simulado y produce el insumo que necesita el operador europeo.

Figura 1

Diagrama de contexto (C4, nivel 1)



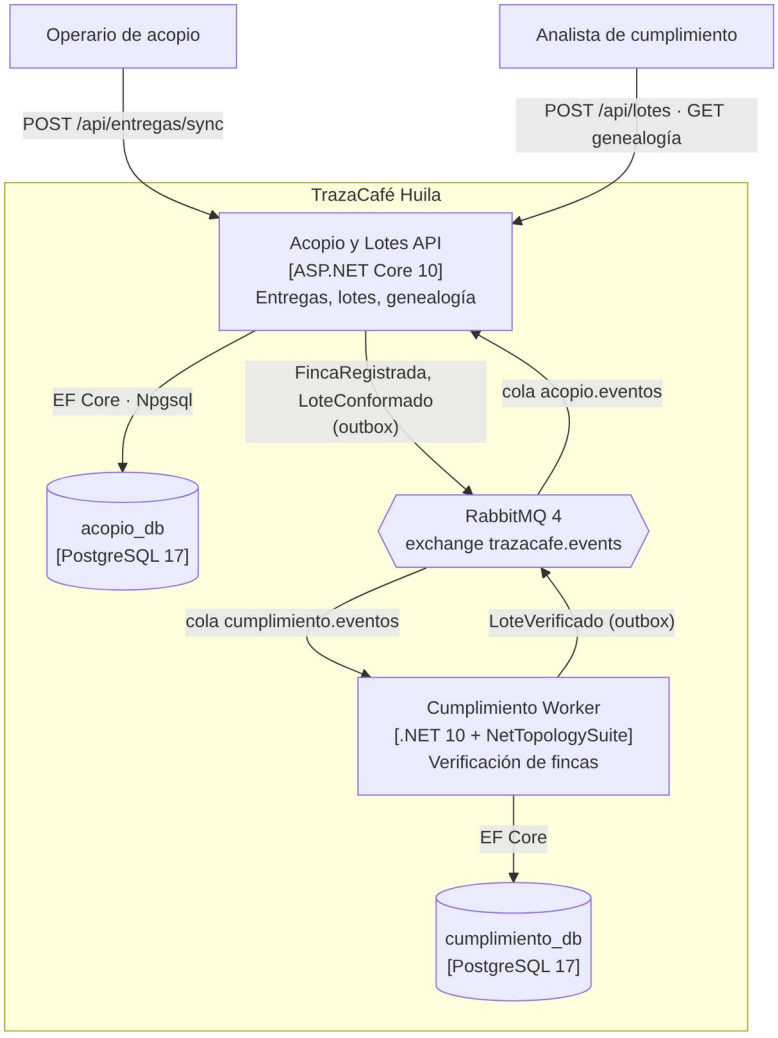
Nota. Elaboración propia. El proveedor geoespacial está simulado en el MVP; el operador en la Unión Europea queda por fuera del alcance.

La Figura 2 baja al nivel de contenedores. La API de Acopio y Lotes escribe en acopio_db y publica eventos en RabbitMQ por medio del outbox. El worker de

Cumplimiento los consume, consulta su réplica de fincas y publica LoteVerificado por el mismo mecanismo.

Figura 2

Diagrama de contenedores (C4, nivel 2)



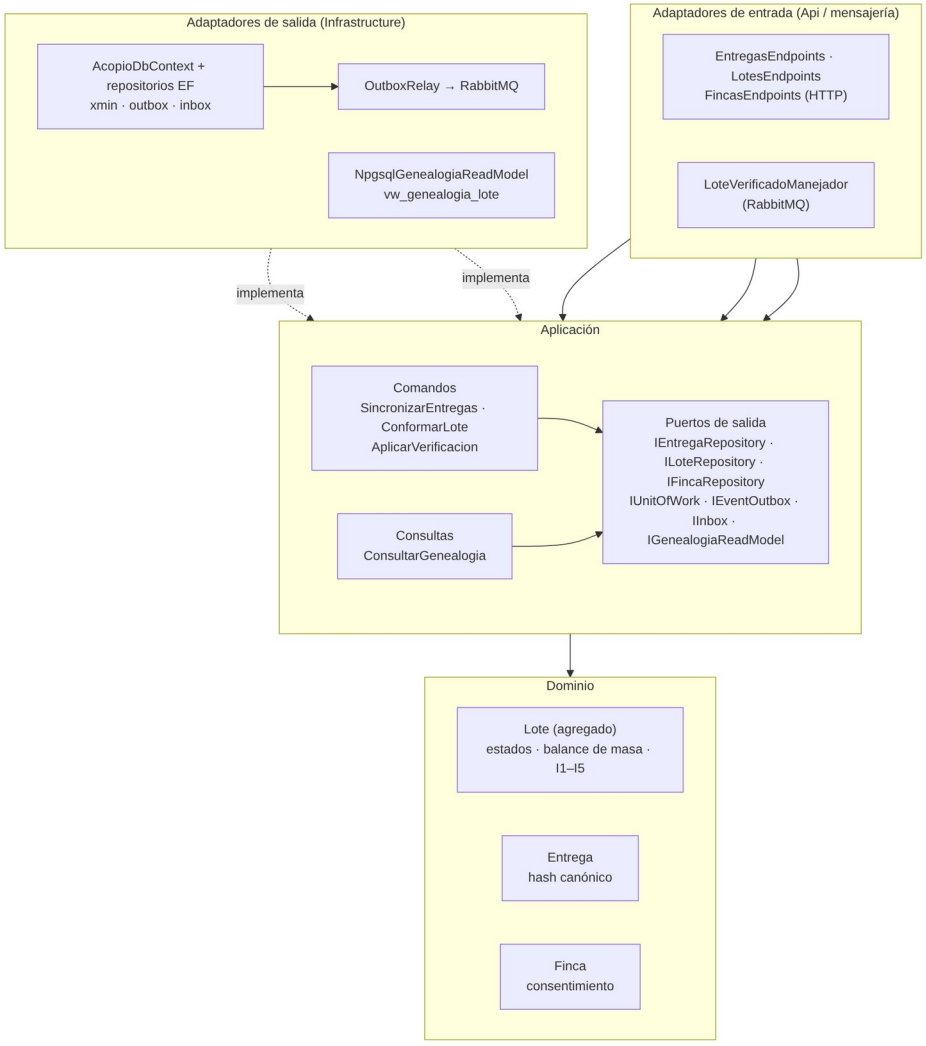
Nota. Elaboración propia.

La Figura 3 presenta los componentes de Acopio y Lotes organizados como hexágono. Las dependencias apuntan hacia adentro: los adaptadores de entrada

llaman a los casos de uso, estos usan el dominio y los puertos, y los adaptadores de salida implementan esos puertos. La regla no depende de la disciplina del equipo, porque cada capa es un proyecto aparte y el compilador impide que el dominio referencie la infraestructura.

Figura 3

Componentes del servicio Acopio y Lotes (arquitectura hexagonal y CQRS ligero)

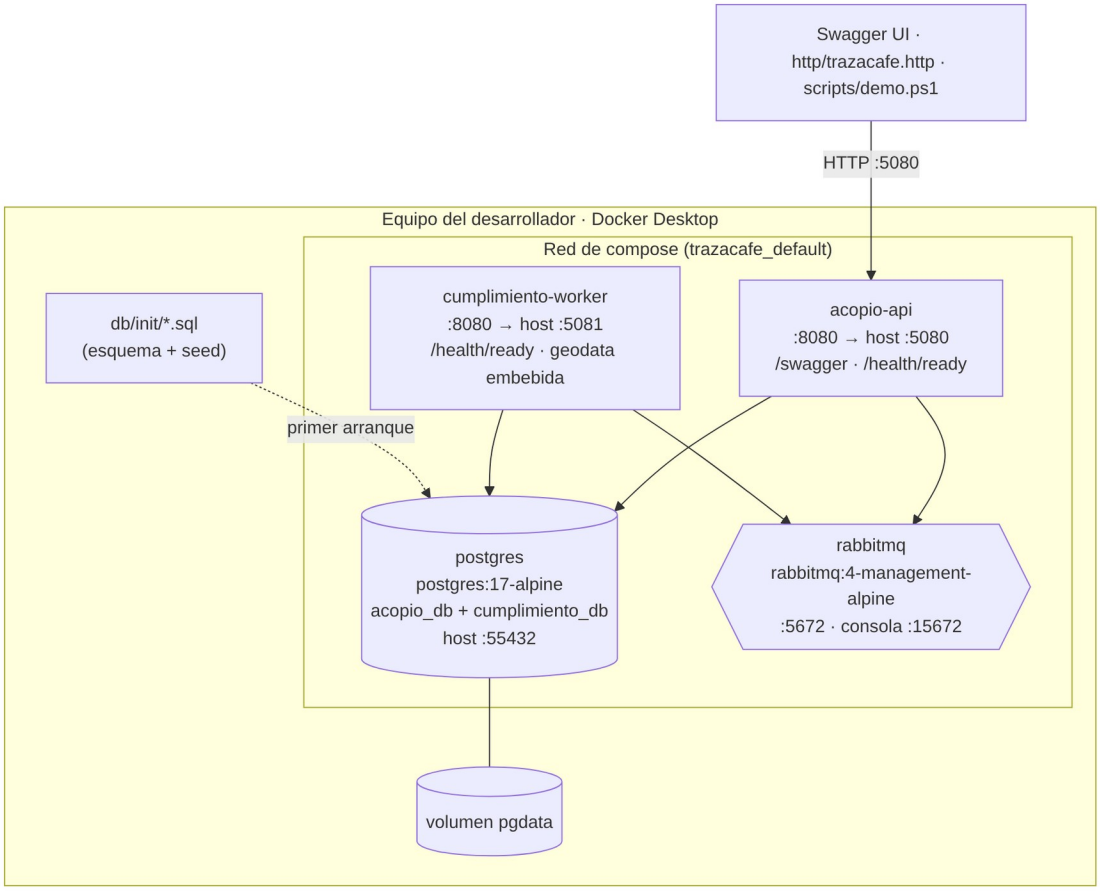


Nota. Elaboración propia. Las flechas punteadas indican que un adaptador implementa un puerto.

La Figura 4 muestra el despliegue local: cuatro contenedores, una sola instancia de PostgreSQL con dos bases lógicas y scripts SQL versionados que crean el esquema y los datos de prueba en el primer arranque.

Figura 4

Diagrama de despliegue local con Docker Compose

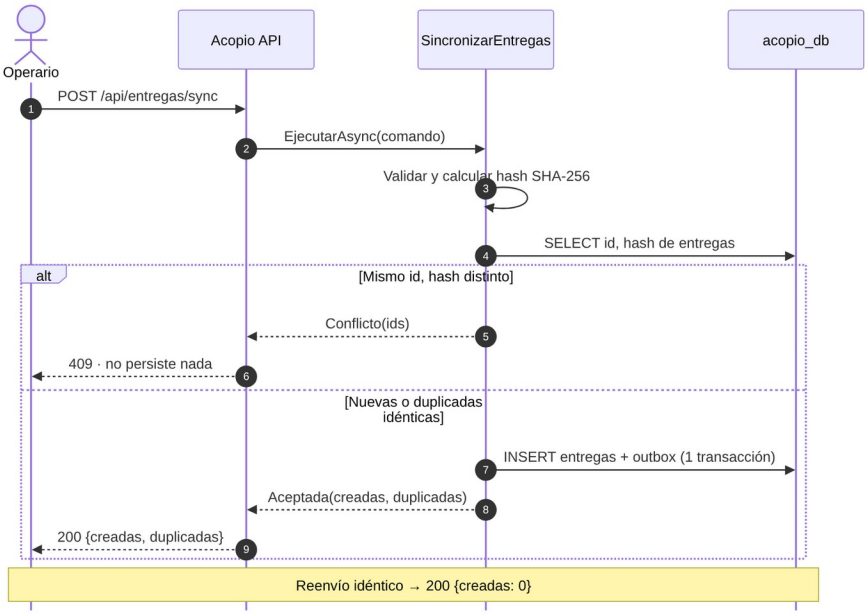


Nota. Elaboración propia.

Las Figuras 5 y 6 describen los dos flujos que sostienen QA1 y QA2. En la sincronización, el servicio compara el hash de cada entrega con el registrado: si coincide, la entrega se reporta como duplicada; si difiere, la solicitud completa se rechaza con 409. En la verificación, la API responde 202 de inmediato y el estado final llega después, cuando Cumplimiento publica su resultado.

Figura 5

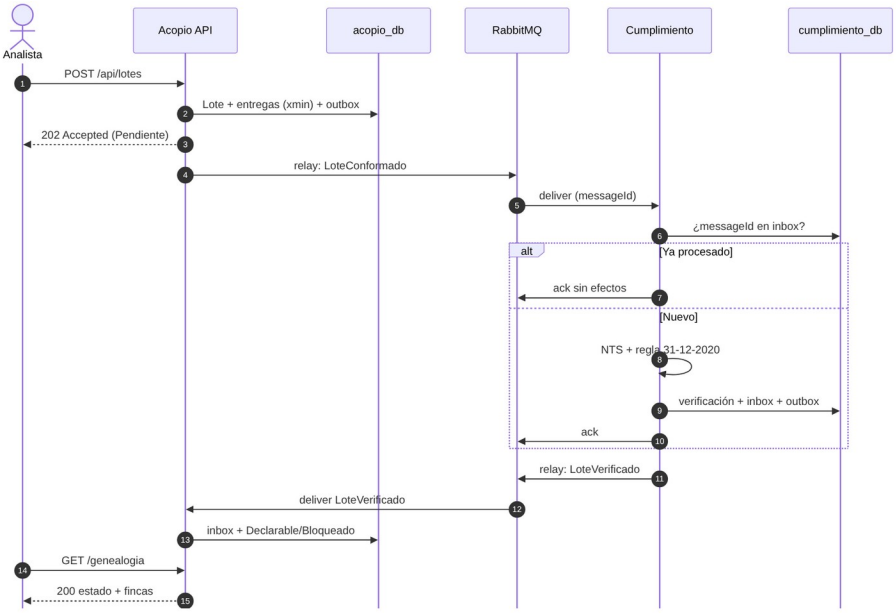
Secuencia de la sincronización idempotente de entregas



Nota. Elaboración propia.

Figura 6

Secuencia de la conformación y verificación asíncrona del lote



Nota. Elaboración propia. Si una finca aún no está replicada, el mensaje pasa a la cola de reintento (TTL de 5 s) y, tras cinco intentos, a la cola de mensajes muertos.

Decisiones arquitectónicas

Registramos seis decisiones en formato ADR, con contexto, decisión, alternativas y consecuencias (Nygard, 2011). La Tabla 2 las resume; el texto completo está en la carpeta docs/adr del repositorio.

Tabla 2

Resumen de los registros de decisiones arquitectónicas (ADR)

ADR	Decisión	Alternativas descartadas	Consecuencia principal
001 · Estilo	Dos servicios hexagonales orientados a eventos con CQRS ligero. Plan B: monolito modular.	Tres servicios; monolito desde el inicio; Event Sourcing.	Aislamiento de fallas a cambio de consistencia eventual.
002 · Mensajería	RabbitMQ con Transactional Outbox e Inbox; reintentos con TTL y DLQ; topología declarada como código.	Kafka; publicar después del commit; CDC con Debezium.	Entrega al menos una vez; efectos exactamente una vez gracias al inbox.
003 · Réplica	Cumplimiento mantiene una réplica de geometrías alimentada por FincaRegistrada.	Consulta HTTP síncrona a Acopio; geometrías dentro de cada evento de lote.	Autonomía a cambio de datos duplicados y eventualmente consistentes.
004 · Ciclo de vida	Estados PendienteVerificacion, Declarable y Bloqueado; el POST responde 202.	Verificación síncrona dentro del POST.	Ventana de inconsistencia visible y medible.
005 · Integridad	UUID del cliente con hash SHA-256; concurrencia optimista con xmin.	Idempotency-Key por petición; bloqueo pesimista.	409 ante conflictos; el cliente reintenta con datos frescos.
006 · Geoespacial	NetTopologySuite en memoria detrás de un puerto; regla de corte en el dominio.	PostGIS; API externa de monitoreo forestal.	Solo capas que caben en memoria; PostGIS como evolución.

Nota. Elaboración propia.

Tecnologías

Cada tecnología responde a un driver o a una restricción. La Tabla 3 resume la elección y su justificación.

Tabla 3

Tecnologías del prototipo y justificación

Tecnología	Uso	Justificación
.NET 10 (LTS), ASP.NET Core Minimal APIs	API y worker	Versión con soporte de largo plazo vigente; .NET 8 y .NET 9 dejan de recibir soporte el 10 de noviembre de 2026 (Bhandari, 2026).
EF Core 10 y Npgsql	Escritura	Unidad de trabajo que confirma en una transacción los cambios, el outbox y el inbox; concurrencia optimista con xmin (Npgsql, s.f.).
Npgsql directo (ADO.NET)	Lectura	La consulta de genealogía no necesita el dominio ni el seguimiento de cambios del ORM.
PostgreSQL 17	Persistencia	FOR UPDATE SKIP LOCKED para el relay del outbox, jsonb para geometrías y eventos, y llaves foráneas diferibles.
RabbitMQ 4	Mensajería	Confirmaciones del publicador, ack manual y dead-lettering (RabbitMQ, s.f.); más liviano que Kafka para este volumen.
NetTopologySuite	Verificación geoespacial	Intersección de polígonos e índice espacial STRtree sin infraestructura adicional.
xUnit	Pruebas	El dominio y los casos de uso se prueban sin infraestructura gracias a los puertos.
OpenAPI y Swagger UI	Contrato de la API	Documentación navegable para la demostración.
Docker Compose	Despliegue local	Todo el entorno se levanta con un solo comando.

Nota. Elaboración propia.

Implementación

Alcance del MVP y definición de terminado

Antes de escribir código fijamos el alcance en tres casos de uso de punta a punta: sincronizar entregas, conformar un lote con verificación asíncrona y consultar su genealogía. Las fincas se cargan como datos de prueba y no cuentan como caso de uso. La definición de terminado también quedó escrita: el entorno completo se levanta

con docker compose up; las pruebas pasan; el script de demostración muestra las respuestas 200, 409, 202 y 422 esperadas y los tres estados finales de los lotes; y los endpoints /health/ready de ambos servicios responden Healthy.

Una estructura que hace visible el patrón

La organización del repositorio refleja la arquitectura. Cada servicio tiene cuatro proyectos (Domain, Application, Infrastructure y Api o Worker) y, dentro de Application, los comandos y las consultas están en carpetas separadas. Los eventos de integración viven en un proyecto propio sin dependencias, que funciona como lenguaje publicado entre los dos contextos.

```
src/
  TrazaCafe.Contracts/               eventos de integración versionados
  BuildingBlocks/                    outbox, inbox, RabbitMQ, salud
  Acopio/
    TrazaCafe.Acopio.Domain/         Finca, Entrega, Lote (agregado)
    TrazaCafe.Acopio.Application/    Puertos/  Comandos/  Consultas/
    TrazaCafe.Acopio.Infrastructure/ EF Core, lectura con Npgsql
    TrazaCafe.Acopio.Api/            Minimal APIs, ProblemDetails, OpenAPI
  Cumplimiento/
    ...Domain/  ...Application/  ...Infrastructure/  ...Worker/
tests/   db/init/   geodata/   docs/adr/
```

Sincronización idempotente

La sincronización construye primero todas las entregas candidatas; el dominio valida cada una y calcula su hash. Luego las compara con lo registrado y decide en bloque, de modo que una sola entrega alterada impide guardar el resto de la solicitud:

```
var hashesPrevios = await entregas.ObtenerHashesAsync(ids, ct);
var enConflicto = candidatas
    .Where(e => hashesPrevios.TryGetValue(e.Id, out var previo) && previo != e.Hash)
    .Select(e => e.Id).ToList();
// 409: si hay un solo conflicto, no se guarda nada de la solicitud
if (enConflicto.Count > 0)
    return new ResultadoSincronizacion.Conflicto(enConflicto);

foreach (var entrega in candidatas.Where(e => !hashesPrevios.ContainsKey(e.Id)))
{
    entregas.Agregar(entrega);
    outbox.Encolar(TiposDeEvento.EntregaRegistrada, /* evento */);
}
// entregas + outbox en una sola transacción
await unitOfWork.GuardarCambiosAsync(ct);
```

Si dos reenvíos idénticos llegan al mismo tiempo, la llave primaria de la tabla resuelve la carrera y la API reintenta una vez; en el segundo intento las entregas aparecen como duplicadas. La Figura 7 muestra el comportamiento con el script de demostración.

Figura 7

Sincronización idempotente y conflicto por contenido alterado

```
═══ 3. QA1 · Mismo entregaId con kilos alterados ═══
HTTP 409 · Entregas en conflicto: e1000000-0000-4000-8000-000000000001
```

Nota. Captura propia del prototipo en ejecución.

Outbox, consumidor idempotente y reintentos

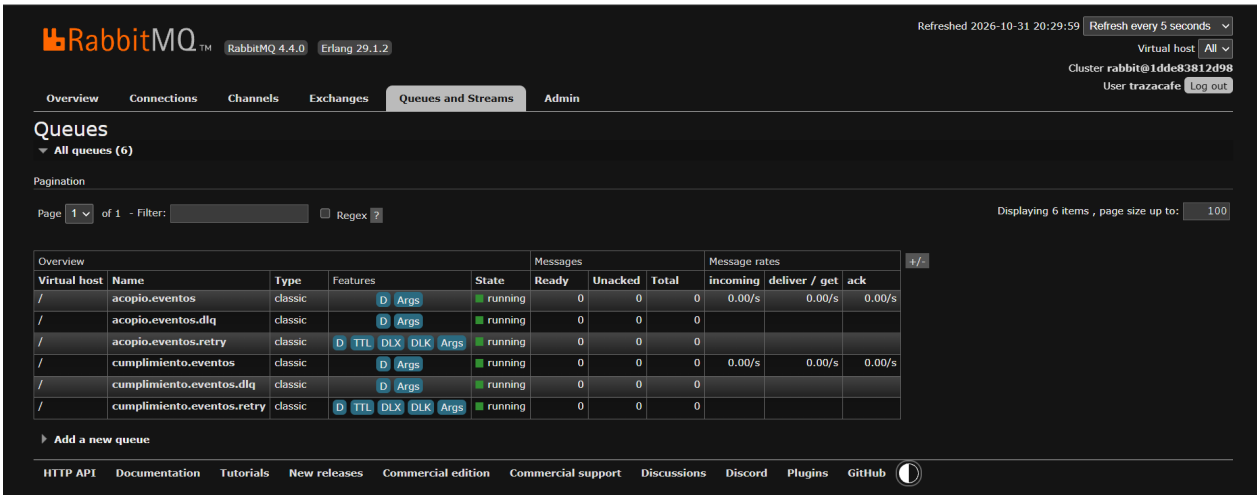
El relay del outbox es un servicio en segundo plano que cada segundo toma los eventos pendientes, los publica con confirmación del broker y los marca como publicados dentro de la misma transacción. La cláusula FOR UPDATE SKIP LOCKED permite, en producción, ejecutar varias réplicas sin que dos de ellas publiquen el mismo evento al mismo tiempo:

```
SELECT * FROM outbox_messages
WHERE publicado_en IS NULL
ORDER BY ocurrido_en, id
LIMIT 50
FOR UPDATE SKIP LOCKED
```

Del lado del consumidor, cada caso de uso consulta primero el inbox: si el messageId ya existe, el mensaje se confirma sin efectos. Cuando el procesamiento falla, el mensaje pasa a una cola de reintento con un TTL de 5 segundos que lo devuelve a la cola principal y, después de cinco intentos, termina en la cola de mensajes muertos (Figura 8). En la revisión del código apareció un riesgo que el diseño no había previsto: si la API arrancaba antes que el worker, los eventos FincaRegistrada de los datos de prueba se publicaban hacia un exchange sin la cola suscriptora y RabbitMQ los descartaba sin reportar error. Lo resolvimos declarando la topología completa como código desde cualquier servicio que arranque (ADR-002).

Figura 8

Topología del broker: colas principales, de reintento y de mensajes muertos



Nota. Captura propia del prototipo en ejecución.

La regla de la fecha de corte en el dominio

El adaptador geoespacial solo informa qué áreas con pérdida de bosque intersectan la finca y en qué fecha ocurrió cada pérdida. La decisión regulatoria la toma el dominio:

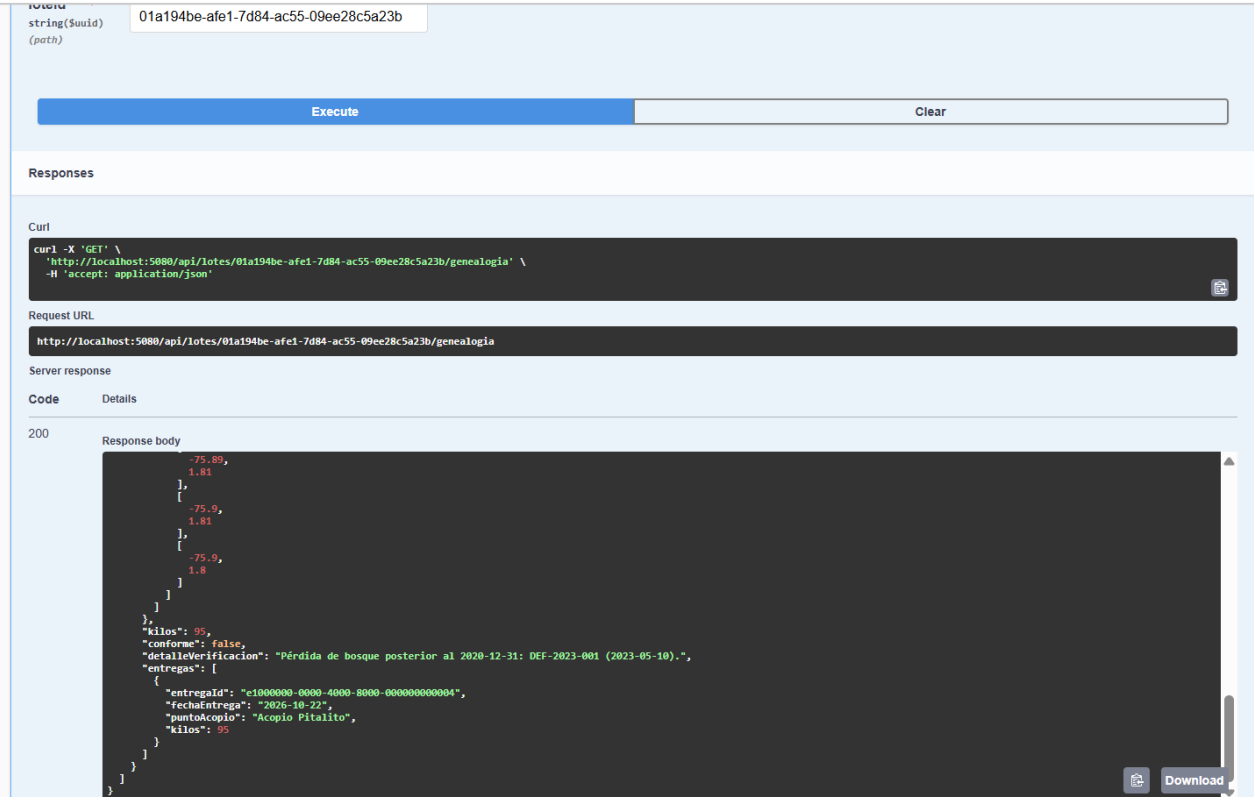
```
public static readonly DateOnly FechaDeCorte = new(2020, 12, 31);
public static bool EsPosteriorAlCorte(DateOnly fechaPerdida)
    => fechaPerdida > FechaDeCorte;

// ResultadoFinca.Evaluar(fincaId, perdidas)
var posteriores = perdidas
    .Where(p => ReglaEudr.EsPosteriorAlCorte(p.FechaPerdida)).ToList();
return new ResultadoFinca
    { FincaId = fincaId, Conforme = posteriores.Count == 0, Detalle = detalle };
```

Esa separación permite que los datos de prueba pongan la regla a prueba. La finca Buenavista intersecta un área con pérdida en 2019, anterior al corte, y su lote resulta Declarable; El Mirador intersecta una pérdida de 2023 y su lote queda Bloqueado (Figura 9).

Figura 9

Genealogía de un lote bloqueado



Nota. Captura propia del prototipo en ejecución.

Concurrencia optimista

Para que dos analistas no asignen la misma entrega a lotes distintos, usamos la columna de sistema xmin de PostgreSQL como token de concurrencia. EF Core agrega la condición WHERE xmin = @original a cada actualización; si otra transacción modificó la fila, el UPDATE afecta cero filas y la API responde 409. Comprobamos el mecanismo directamente en SQL antes de integrarlo.

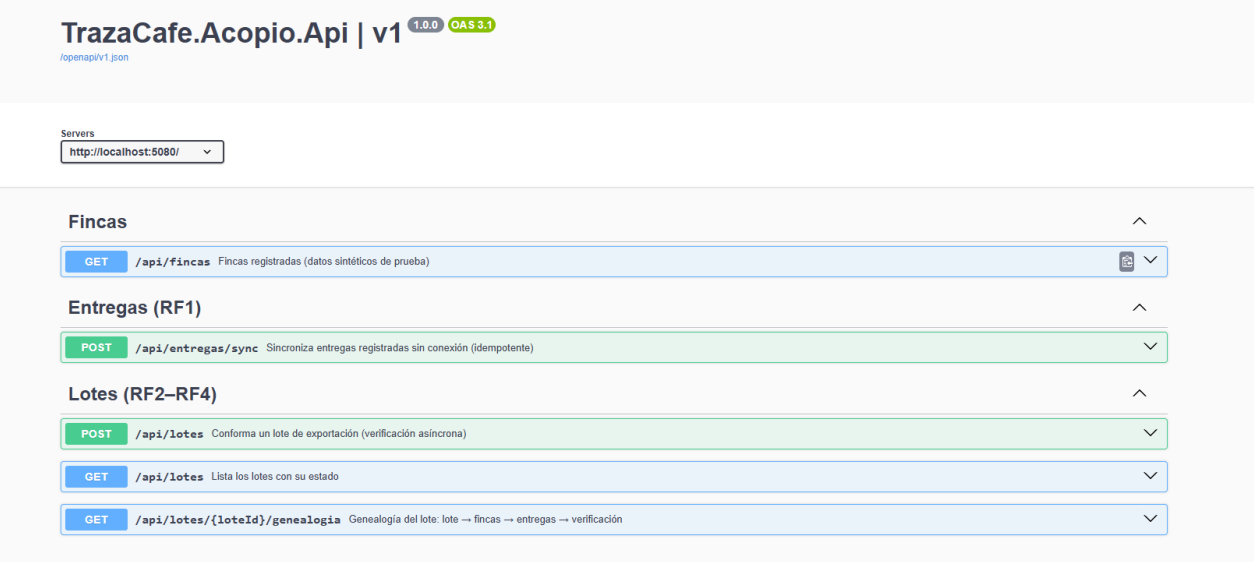
```
b.Property<uint>("Version")
    .HasColumnName("xmin")
    .HasColumnType("xid")
    .ValueGeneratedOnAddOrUpdate()
    .IsConcurrencyToken();
```

Calidad transversal

Los errores se responden con ProblemDetails (RFC 9457): 400 para datos inválidos, 404 para recursos inexistentes, 409 para conflictos y 422 para reglas de negocio. Los registros se escriben en una línea por evento, con el messageId y el tipo de evento como contexto, lo que permite seguir un mensaje entre servicios. Ambos servicios exponen /health/live y /health/ready, que comprueban PostgreSQL, RabbitMQ y, en Cumplimiento, que la capa geoespacial se haya cargado. La API publica su contrato con OpenAPI y Swagger UI (Figura 10), y el repositorio incluye un archivo .http con el recorrido de la demostración. La suite tiene 34 pruebas automatizadas que cubren las invariantes del agregado, el hash canónico, la regla de corte, la idempotencia de la sincronización, el inbox ante mensajes repetidos y el adaptador de NetTopologySuite (Figura 11).

Figura 10

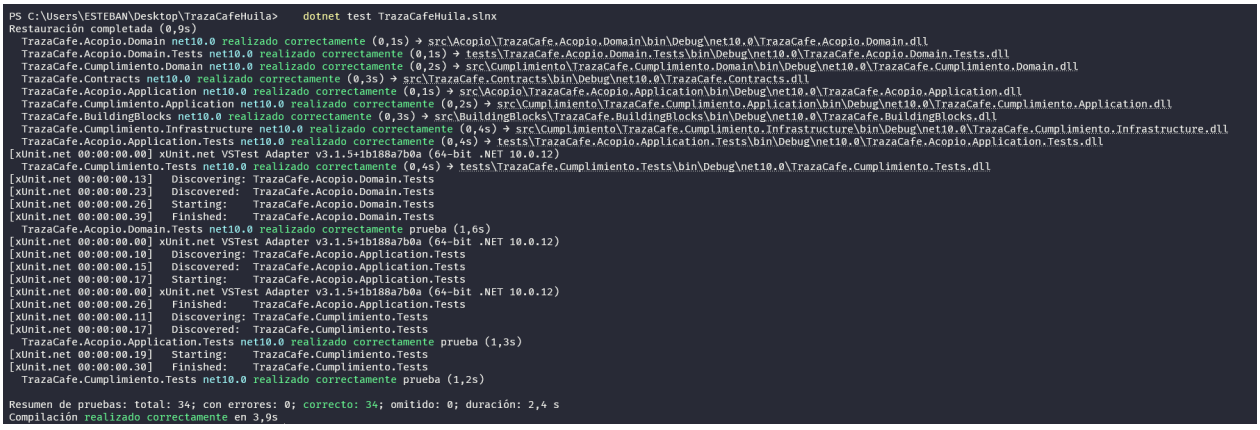
Contrato de la API en Swagger UI



Nota. Captura propia del prototipo en ejecución.

Figura 11

Resultado de la suite de pruebas automatizadas



```
PS C:\Users\ESTEBAN\Desktop\TrazaCafeHuila>    dotnet test TrazaCafeHuila.slnx
Restauración completada (0,9s)
  TrazaCafe.Acopio.Domain net10.0 realizado correctamente (0,1s) → src\Acopio\TrazaCafe.Acopio.Domain\bin\Debug\net10.0\TrazaCafe.Acopio.Domain.dll
  TrazaCafe.Acopio.Domain.Tests net10.0 realizado correctamente (0,1s) → tests\TrazaCafe.Acopio.Domain.Tests\bin\Debug\net10.0\TrazaCafe.Acopio.Domain.Tests.dll
  TrazaCafe.Cumplimiento.Domain net10.0 realizado correctamente (0,2s) → src\Cumplimiento\TrazaCafe.Cumplimiento.Domain\bin\Debug\net10.0\TrazaCafe.Cumplimiento.Domain.dll
  TrazaCafe.Contracts net10.0 realizado correctamente (0,3s) → src\TrazaCafe.Contracts\bin\Debug\net10.0\TrazaCafe.Contracts.dll
  TrazaCafe.Acopio.Application net10.0 realizado correctamente (0,1s) → src\Acopio\TrazaCafe.Acopio.Application\bin\Debug\net10.0\TrazaCafe.Acopio.Application.dll
  TrazaCafe.Cumplimiento.Application net10.0 realizado correctamente (0,2s) → src\Cumplimiento\TrazaCafe.Cumplimiento.Application\bin\Debug\net10.0\TrazaCafe.Cumplimiento.Application.dll
  TrazaCafe.BuildingBlocks net10.0 realizado correctamente (0,3s) → src\BuildingBlocks\TrazaCafe.BuildingBlocks\bin\Debug\net10.0\TrazaCafe.BuildingBlocks.dll
  TrazaCafe.Cumplimiento.Infrastructure net10.0 realizado correctamente (0,4s) → src\Cumplimiento\TrazaCafe.Cumplimiento.Infrastructure\bin\Debug\net10.0\TrazaCafe.Cumplimiento.Infrastructure.dll
  TrazaCafe.Acopio.Application.Tests net10.0 realizado correctamente (0,4s) → tests\TrazaCafe.Acopio.Application.Tests\bin\Debug\net10.0\TrazaCafe.Acopio.Application.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.5+1b188a7b0a (64-bit .NET 10.0.12)
  TrazaCafe.Cumplimiento.Tests net10.0 realizado correctamente (0,4s) → tests\TrazaCafe.Cumplimiento.Tests\bin\Debug\net10.0\TrazaCafe.Cumplimiento.Tests.dll
[xUnit.net 00:00:00.13]   Discovering: TrazaCafe.Acopio.Domain.Tests
[xUnit.net 00:00:00.23]   Discovered:  TrazaCafe.Acopio.Domain.Tests
[xUnit.net 00:00:00.26]   Starting:    TrazaCafe.Acopio.Domain.Tests
[xUnit.net 00:00:00.39]   Finished:    TrazaCafe.Acopio.Domain.Tests
  TrazaCafe.Acopio.Domain.Tests net10.0 realizado correctamente prueba (1,6s)
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.5+1b188a7b0a (64-bit .NET 10.0.12)
[xUnit.net 00:00:00.10]   Discovering: TrazaCafe.Acopio.Application.Tests
[xUnit.net 00:00:00.15]   Discovered:  TrazaCafe.Acopio.Application.Tests
[xUnit.net 00:00:00.17]   Starting:    TrazaCafe.Acopio.Application.Tests
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.5+1b188a7b0a (64-bit .NET 10.0.12)
[xUnit.net 00:00:00.26]   Finished:    TrazaCafe.Acopio.Application.Tests
[xUnit.net 00:00:00.11]   Discovering: TrazaCafe.Cumplimiento.Tests
[xUnit.net 00:00:00.17]   Discovered:  TrazaCafe.Cumplimiento.Tests
  TrazaCafe.Acopio.Application.Tests net10.0 realizado correctamente prueba (1,3s)
[xUnit.net 00:00:00.19]   Starting:    TrazaCafe.Cumplimiento.Tests
[xUnit.net 00:00:00.30]   Finished:    TrazaCafe.Cumplimiento.Tests
  TrazaCafe.Cumplimiento.Tests net10.0 realizado correctamente prueba (1,2s)

Resumen de pruebas: total: 34; con errores: 0; correcto: 34; omitido: 0; duración: 2,4 s
Compilación realizado correctamente en 3,9s
```

Nota. Captura propia.

Evaluación y comparación con alternativas

Resultados medidos

Medimos cada escenario en el MVP en lugar de dar los atributos por cumplidos. Todas las mediciones se hicieron en un solo equipo con Docker Desktop sobre Windows, desde el cliente y con datos sintéticos; indican un orden de magnitud y no el desempeño en producción (Tabla 4).

Tabla 4

Resultados medidos en el prototipo

Escenario	Resultado	Meta	Método y muestra
QA1 · Reenvío idéntico	0 kilos y 0 eventos duplicados	0 duplicados	scripts/demo.ps1 (3 corridas) y prueba automatizada.
QA1 · Contenido alterado	409; no se guardó nada	409 atómico	scripts/demo.ps1 y prueba automatizada.
QA2 · Operación normal	Mediana 1,50 s; rango 0,85 s a 1,96 s	≤ 10 s	n = 7 lotes en 3 corridas; diferencia entre conformadoEn y verificadoEn.
QA2 · Worker detenido	0 eventos perdidos; 6 de 6 lotes resueltos tras esperar 40 s y 201 s en la cola	0 perdidos; ≤ 10 s tras recuperar	2 corridas, n = 6 lotes; worker detenido durante la conformación; consulta 10 s después de reiniciarlo.
QA3 · Genealogía de 200 entregas	p50 = 40,6 ms; p95 = 52,5 ms; máx. = 89,2 ms	< 500 ms	n = 50 peticiones; se excluyeron 5 peticiones previas de calentamiento.
Sincronización de 200 entregas	193 ms	Sin meta	1 petición.
QA4 · Modificabilidad	Domain y Application sin dependencias de paquetes	0 cambios en el núcleo	Revisión de los proyectos y prueba del adaptador geoespacial.

Nota. Elaboración propia. Mediciones locales, desde el cliente y con datos sintéticos; no constituyen una prueba de carga.

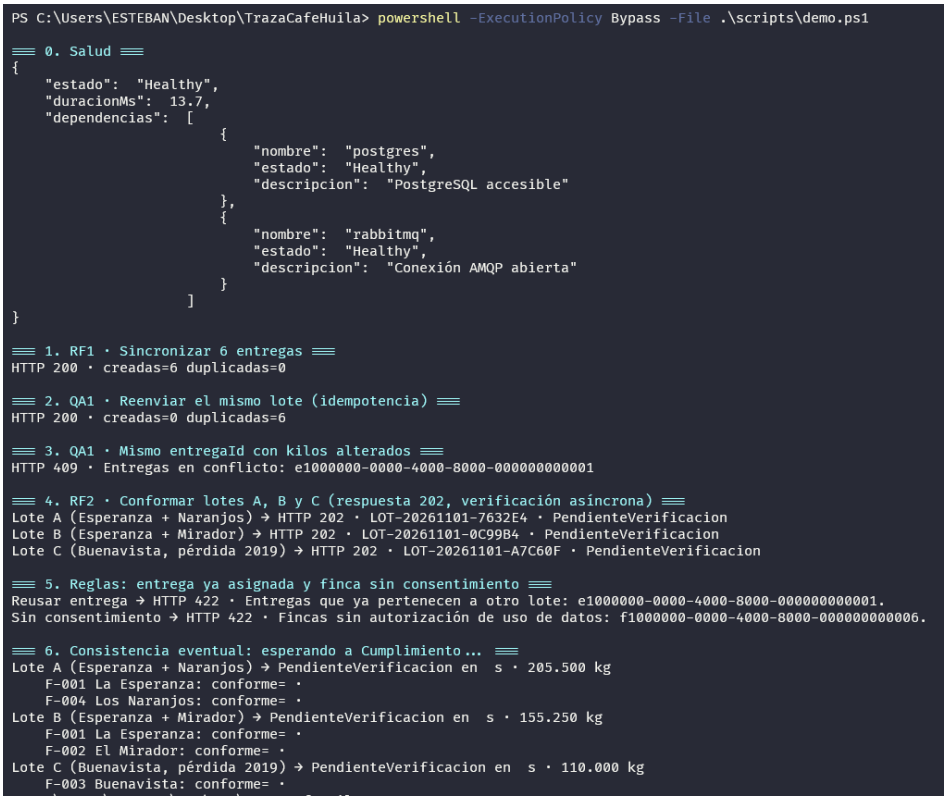
Lo que demuestra la prueba de resiliencia

La prueba de QA2 merece una precisión. Que no se perdiera ningún evento con el worker detenido es mérito del outbox y del inbox, no de tener dos servicios: un monolito modular con outbox lograría el mismo resultado. Lo que solo demuestra la separación en procesos es que la API siguió aceptando solicitudes y respondiendo 202 mientras Cumplimiento estaba caído, y que los lotes llegaron a su estado final cuando el worker volvió (Figura 12). En la segunda corrida, los eventos permanecieron 201 segundos en la cola y los tres lotes alcanzaron su estado final menos de 10 segundos después de reiniciar el worker. En un monolito, un fallo del componente geoespacial, por ejemplo un consumo excesivo de memoria al cargar una capa grande, detendría también la API. Esto último es un argumento y no una medición, porque no implementamos el monolito para comprobarlo.

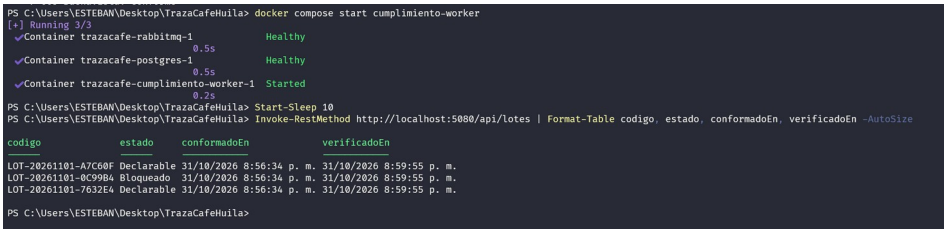
Figura 12

Prueba de resiliencia con el worker de Cumplimiento detenido

(a) Worker de Cumplimiento detenido



(b) Diez segundos después de reiniciar el worker



Nota. Captura propia del prototipo en ejecución.

Comparación con alternativas

Comparamos el estilo elegido con un monolito modular (las mismas capas hexagonales, eventos en proceso y outbox) y con una solución serverless (Azure Functions con Azure Service Bus). Calificamos ocho criterios de 1 a 5, donde 5 es siempre la opción favorable; en complejidad, costo operativo y curva de aprendizaje, 5 significa más simple o más barato (Tabla 5). Los puntajes son valoraciones cualitativas apoyadas en Richards y Ford (2020); salvo el aislamiento de fallas de la alternativa elegida, ninguno proviene de una medición.

Tabla 5

Matriz comparativa simple (escala de 1 a 5)

Criterio	Dos servicios EDA (elegido)	Monolito modular	Serverless
Complejidad	2	4	3
Escalabilidad	4	3	5
Mantenibilidad	4	4	3
Testabilidad	4	5	3
Costo operativo	2	5	4
Curva de aprendizaje	2	4	3
Time-to-market	3	5	3
Aislamiento de fallas	5 ‡	2 †	4 †
Total	**26**	**32**	**28**

Nota. Elaboración propia. † Puntaje argumentado, no medido. ‡ Respaldado por la prueba de resiliencia del prototipo.

Para la versión ponderada definimos los pesos antes de calcular, a partir de los drivers de la fase de arquitectura (Tabla 6). QA1 y QA3 no reciben un criterio propio porque se logran con mecanismos que no dependen del estilo de despliegue

(idempotencia, outbox y una vista SQL) y podrían implementarse igual en las tres alternativas; es decir, no discriminan entre ellas. QA2 se representa con el aislamiento de fallas y QA4 con la mantenibilidad. La testabilidad recoge que las medidas de QA1 y QA4 se verifican con pruebas automatizadas. La restricción de un desarrollador y 24 horas se reparte entre time-to-market, complejidad y curva de aprendizaje. Escalabilidad y costo operativo no tienen escenario en la fase de arquitectura y reciben el menor peso. En total, los drivers de calidad suman 50 %, la restricción 35 % y los criterios sin driver 15 %.

Tabla 6

Pesos de los criterios y su origen

Criterio	Peso (%)	Origen
Aislamiento de fallas	25	QA2, el driver de calidad de mayor prioridad que sí distingue entre alternativas.
Mantenibilidad	15	QA4 (modificabilidad).
Testabilidad	10	Las medidas de QA1 y QA4 se verifican con pruebas automatizadas.
Time-to-market	15	Restricción: un desarrollador y cerca de 24 horas.
Complejidad	10	Restricción: un desarrollador y cerca de 24 horas.
Curva de aprendizaje	10	Restricción: un desarrollador y cerca de 24 horas.
Escalabilidad	10	Sin escenario en la fase de arquitectura; relevante para producción.
Costo operativo	5	Sin escenario; el prototipo corre en local.
Total	**100**	**Drivers de calidad 50 %; restricción 35 %; sin driver 15 %.**

Nota. Elaboración propia. Pesos fijados antes del cálculo. QA1 y QA3 no reciben criterio porque no discriminan entre alternativas.

Tabla 7

Matriz comparativa ponderada (contribución = puntaje × peso / 100)

Criterio (peso)	Dos servicios EDA (elegido)	Monolito modular	Serverless
Aislamiento de fallas (25)	1,25	0,50	1,00
Mantenibilidad (15)	0,60	0,60	0,45
Testabilidad (10)	0,40	0,50	0,30
Time-to-market (15)	0,45	0,75	0,45
Complejidad (10)	0,20	0,40	0,30
Curva de aprendizaje (10)	0,20	0,40	0,30
Escalabilidad (10)	0,40	0,30	0,50
Costo operativo (5)	0,10	0,25	0,20
Total ponderado (sobre 5)	**3,60**	**3,70**	**3,50**

Nota. Elaboración propia a partir de las Tablas 5 y 6.

El resultado coincide en las dos versiones: el monolito modular queda primero, con 32 puntos en la matriz simple y 3,70 en la ponderada. En la ponderada le sigue el estilo elegido con 3,60 y luego serverless con 3,50. La diferencia entre el monolito y nuestra elección es de 0,10 puntos y depende de los pesos: basta trasladar dos puntos de costo operativo a aislamiento de fallas para que el orden se invierta. Lo leemos como un empate técnico y no como una victoria de ninguna de las dos opciones.

Con estos pesos, la evaluación no confirma que el estilo elegido sea superior; lo matiza. Para el volumen del prototipo, un monolito modular con outbox habría sido una elección igual de defendible y más barata de operar. La separación en dos servicios se justifica solo si el aislamiento de fallas tiene la prioridad que le dimos, y el único dato que respalda ese criterio es la prueba de resiliencia, que corrimos únicamente sobre nuestra alternativa. Si tuviéramos que decidir de nuevo, evaluaríamos un punto

intermedio: un monolito modular con el verificador geoespacial como proceso aparte, que conserva el aislamiento con menos piezas. Esta lectura coincide con la recomendación de no empezar por microservicios mientras los límites del dominio y la carga no lo exijan (Fowler, 2015; Newman, 2021).

Desafíos y riesgos observados

Además de la carrera de arranque descrita en la implementación, aparecieron otros desafíos. El primero fue la escritura doble: guardar y publicar son dos operaciones y, sin outbox, un fallo entre ambas pierde el evento. Aun así, el outbox por sí solo no evita duplicados; garantiza la entrega al menos una vez, y son el inbox y la idempotencia los que producen efectos exactamente una vez (Kleppmann, 2017). El segundo fue el orden de escritura en EF Core: como Entrega y Lote no tienen navegaciones entre sí, el ORM no garantiza que el lote se inserte antes de actualizar las entregas, así que declaramos las llaves foráneas como diferibles para que se validen al confirmar la transacción. El tercero tiene que ver con la experiencia de usuario: responder 202 obliga al cliente a consultar el estado, por eso hicimos visible la ventana de inconsistencia con el campo segundosHastaVerificacion. Por último, el costo operativo es real: son cuatro contenedores y, sin trazas distribuidas, seguir un mensaje entre servicios depende de buscar su messageId en los registros.

Cuándo no usar este patrón

No recomendaríamos este estilo cuando no hay una necesidad clara de aislar fallas o de escalar partes del sistema por separado, ni cuando el negocio exige consistencia inmediata entre contextos, como un débito financiero que no admite un

estado pendiente. Tampoco conviene en equipos sin experiencia en operar un broker, colas de mensajes muertos y monitoreo distribuido, ni cuando los límites del dominio todavía son inciertos: partir el sistema demasiado pronto fija fronteras que después cuesta mover. Con volúmenes bajos y estables, el costo fijo del broker y de los servicios siempre encendidos difícilmente se recupera.

Hoja de ruta hacia producción

Las limitaciones del prototipo son conocidas: datos sintéticos, sin autenticación, una sola instancia de PostgreSQL, verificación en memoria, outbox sin purga y sin trazas distribuidas. La Tabla 8 las organiza en una hoja de ruta hacia producción de cuatro semanas para un desarrollador a tiempo completo. En la nube, el camino natural sería Azure Container Apps para los servicios y Azure Database for PostgreSQL para los datos.

Tabla 8

Hoja de ruta hacia producción

Semana	Actividades principales	Estado en el prototipo
1 · Arquitectura y esqueleto	Integración continua con GitHub Actions; esquemas formales de los eventos; migraciones versionadas.	Parcial: drivers, vistas, ADR y esqueleto listos; sin CI ni esquemas formales.
2 · Núcleo de Acopio	Pruebas de integración con Testcontainers; purga y archivo del outbox.	RF1 y RF2 cubiertos; sin Testcontainers.
3 · Cumplimiento	Verificación con PostGIS (ST_Intersects e índice GiST); trazas y métricas con OpenTelemetry.	RF3 y RF4 cubiertos con NetTopologySuite en memoria; sin trazas.
4 · Endurecimiento y despliegue	Autenticación OIDC y roles; registro del consentimiento del caficultor; prueba de carga con k6; despliegue en Azure.	Mediciones locales de QA2 y QA3; sin seguridad ni nube.

Nota. Elaboración propia.

Conclusiones y aprendizajes

El proyecto confirmó que el valor de un patrón se mide en el problema que resuelve. En TrazaCafé Huila, la idempotencia con hash eliminó los kilos duplicados en todos los reenvíos, el outbox con inbox no perdió ningún evento mientras el verificador estuvo detenido y la consulta de genealogía respondió en 52,5 ms en el percentil 95, muy por debajo de la meta. Son resultados obtenidos en local y con datos sintéticos, y así deben leerse.

El aprendizaje más útil vino de la evaluación. Al ponderar los criterios con los drivers declarados, el monolito modular quedó ligeramente por encima de nuestra elección. No lo esperábamos, pero tiene sentido: la parte costosa del diseño, la asincronía confiable, no depende de partir el sistema en servicios. Separar procesos aporta aislamiento de fallas y escalado independiente, y solo vale la pena si esos atributos pesan lo suficiente en el problema.

También aprendimos que, en un sistema orientado a eventos, la infraestructura hace parte del diseño. La pérdida silenciosa de eventos por el orden de arranque no aparece en un diagrama de componentes; aparece en la topología del broker. Documentar las decisiones en ADR nos obligó a nombrar las alternativas y las consecuencias de cada una, y eso facilitó revisarlas con evidencia en lugar de defenderlas por costumbre.

Como trabajo futuro quedan la migración del verificador a PostGIS, que permitirá comprobar QA4 con un cambio real de adaptador; la autenticación y el registro formal del consentimiento del caficultor; las trazas distribuidas con OpenTelemetry; y una prueba de carga con k6 que reemplace las mediciones puntuales de este informe.

Referencias

Bass, L., Clements, P. y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Bhandari, R. (2026, 29 de junio). *.NET 8 and .NET 9 will reach end of support on November 10, 2026*. .NET Blog. https://devblogs.microsoft.com/dotnet/dotnet-8-9-end-of-support/

Brown, S. (s.f.). *The C4 model for visualising software architecture*. https://c4model.com

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2011, 14 de julio). *CQRS*. martinfowler.com. https://martinfowler.com/bliki/CQRS.html

Fowler, M. (2015, 3 de junio). *MonolithFirst*. martinfowler.com. https://martinfowler.com/bliki/MonolithFirst.html

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.

Npgsql. (s.f.). *Concurrency tokens*. https://www.npgsql.org/efcore/modeling/concurrency.html

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

RabbitMQ. (s.f.). *Consumer acknowledgements and publisher confirms*. https://www.rabbitmq.com/docs/confirms

Reglamento (UE) 2023/1115 del Parlamento Europeo y del Consejo, de 31 de mayo de 2023, relativo a la comercialización en el mercado de la Unión y a la exportación desde la Unión de determinadas materias primas y productos derivados asociados a la deforestación y la degradación forestal, y por el que se deroga el Reglamento (UE) n.º 995/2010. (2023). *Diario Oficial de la Unión Europea*, L 150, 206–247. http://data.europa.eu/eli/reg/2023/1115/oj

Reglamento (UE) 2025/2650 del Parlamento Europeo y del Consejo, de 19 de diciembre de 2025, por el que se modifica el Reglamento (UE) 2023/1115. (2025). *Diario Oficial de la Unión Europea*, L 2025/2650. http://data.europa.eu/eli/reg/2025/2650/oj

Richards, M. y Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.